Here, again, process 0 has control flow that depends on the values in the array, which are all exposed to racing operations. This is generally a typical case where source-DPOR may encounter sleep set blocking, which optimal-DPOR can avoid. If some of the processes with higher indices have not completed their execution, processes with lower indices that are in a sleep set, because of a race with process 0's access, will never be removed, as process 0 will "stop" at a high index and not access the lower indices. Again, an accurate wakeup sequence is necessary to avoid sleep set blocking in this program. We present measurements for the lastzero(N) program in Section 11.

Shared variables:	`int barrier := N + 1, counter1, counter2;`
Process 0:	`inc counter1;`
Process j ($j \in 1..\mathsf{N}$):	`inc counter2;` `wait(barrier);`
Process $\mathsf{N}+1$:	`wait(barrier);` `inc counter1;`

Fig. 12. Pseudocode of the wakeup_stress(N) example.

The third and final program is wakeup_stress(N), whose pseudocode is shown in Fig. 12. Here each of N processes increments `counter2` atomically, and two more processes, 0 and $\mathsf{N}+1$, increment `counter1`, with process $\mathsf{N}+1$ waiting for all N processes to finish before doing its operation. Under the assumption that the `inc` operations are dependent (and `wait` operations are not), this program has $2*(\mathsf{N}!)$ Mazurkiewicz traces, differing in the order in which the N processes increment `counter2` and processes 0 and $\mathsf{N}+1$ increment `counter1`.

Whenever an attempt is made to reverse a race, any chosen process will eventually execute an appropriate `inc` operation, removing any other process from the sleep set, so source-DPOR will never encounter sleep set blocking.

On the other hand, all such `inc` operations are also racing with each other, therefore introducing new branches in a wakeup tree. Particularly for the race between processes 0 and $\mathsf{N}+1$, each of the N! schedules of the N processes (that must all finish to enable process $\mathsf{N}+1$) differs in the happens-before order of the `inc counter2` operations. As a result, the wakeup tree at the point where the first `inc counter1` operation is originally executed will eventually have N! branches.

In Table 2 we show the total memory usage of Concuerror (the stateless model checking tool which, as we describe in the next section, is our implementation vehicle) when exploring the interleavings of wakeup_stress(N). It is evident that the base memory usage of the tool is approximately 22 MB, but as the number of interleavings increases the memory used by wakeup trees, which are present only in optimal-DPOR, starts being noticeable and follows an exponential trend. The timing measurements, however, show that the overhead of maintaining the wakeup trees is negligible, even in this case, and the optimal algorithm has time performance which is very similar to source-DPOR.

In general, the purpose of wakeup trees is to "guide the first steps" of future explorations with enough information to ensure that the only traces explored are Mazurkiewicz traces which are distinct from both all previous explored traces and from each other. As a result, an exponential blowup in the size of wakeup trees can only be the result of still needing to explore exponentially many Mazurkiewicz traces, which in turn means that the exponential worst case kicks in only when the DPOR algorithm needs to do exponential work anyway.